

The Quicksort Algorithm: History, Mechanics, and Computational Complexity

Aidan Perez

September 19, 2026

Abstract

The Quicksort Algorithm is a general sorting algorithm. This paper gives a solid background and understanding of the Quicksort algorithm. The paper will focus on its history, mechanics and strategy, computational complexity, and implementation.

1 History of the Algorithm

Quicksort was developed by Sir Tony Hoare in 1959 and was published in 1961. Quicksort came about because Sir Tony was working on machine translating Russian to English. Since the dictionaries were stored on magnetic tape, it forced him to sort the words of the sentence alphabetically. So, a sorting algorithm was necessary. Interestingly enough, the prestigious Quicksort algorithm was only the second algorithm he came up with to solve this problem [1, 3].

Although Sir Tony had the brilliant idea of the Quicksort algorithm, the truly challenging aspect was developing it. The only programming language he knew, Mercury Autocode, lacked the necessary tools to code Quicksort. And when he tried explaining the algorithm to a colleague after coming up with the idea, his colleague couldn't understand it [1, 3].

Sir Tony's breakthrough was through a language called Algol 60. The difference was recursion was allowed for in Algol 60. He quickly developed and published Quicksort in 1961. And he was able to explain it to his colleague in only a few minutes [1, 3].

2 Algorithm Strategy

The goal of Quicksort is to sort a list of elements. The first step is the partitioning process. First, an element must be chosen as the pivot key. Some programmers make it random or pick a specific element. However, I will choose the last element for this description [5].

Once a pivot key is selected, the list needs to be rearranged. This is done by taking the leftmost element greater than the pivot's value and swapping it with the subsequent element less than or equal to the pivot's value. This repeats until there are no more subsequent elements. The final swap is the pivot key to its correct position [5].

Now, the list is split into left and right sublists. Quicksort is then recursively called on both the left and right sublists. This is done until each sublist has 1 or less elements. Thus, the list is sorted [1, 5].

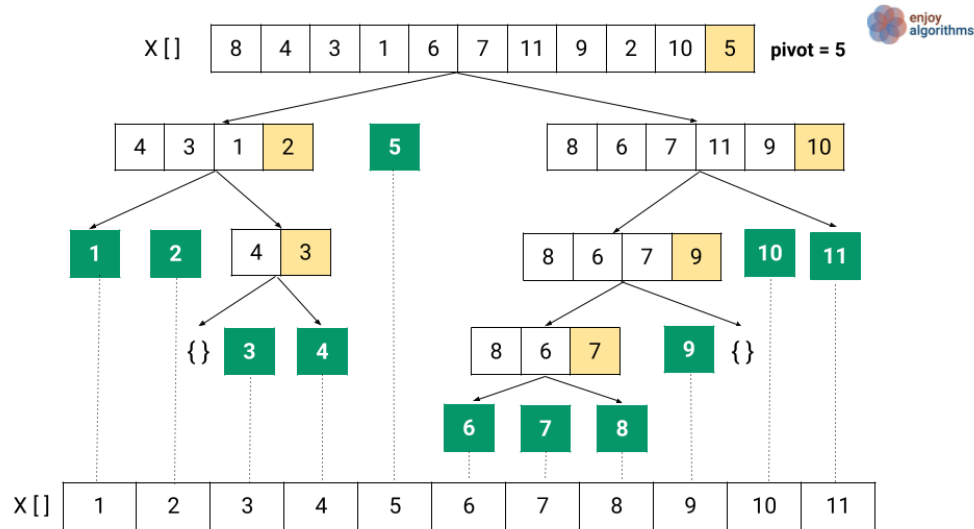


Figure 1: A visual representation of Quicksort [2].

3 Computational Complexity and Mathematical Rigor

For an algorithm, efficiency in time is important. It also helps us define the best use cases for a specific algorithm. So, we will first go over time complexity. For Quicksort, we can separate the time complexity into best, average, and worst cases. The recurrence relation for time complexity can be modeled as [5]:

$$T(n) = T(k) + T(n - k - 1) + c_2n + (c_1 - c_2)$$

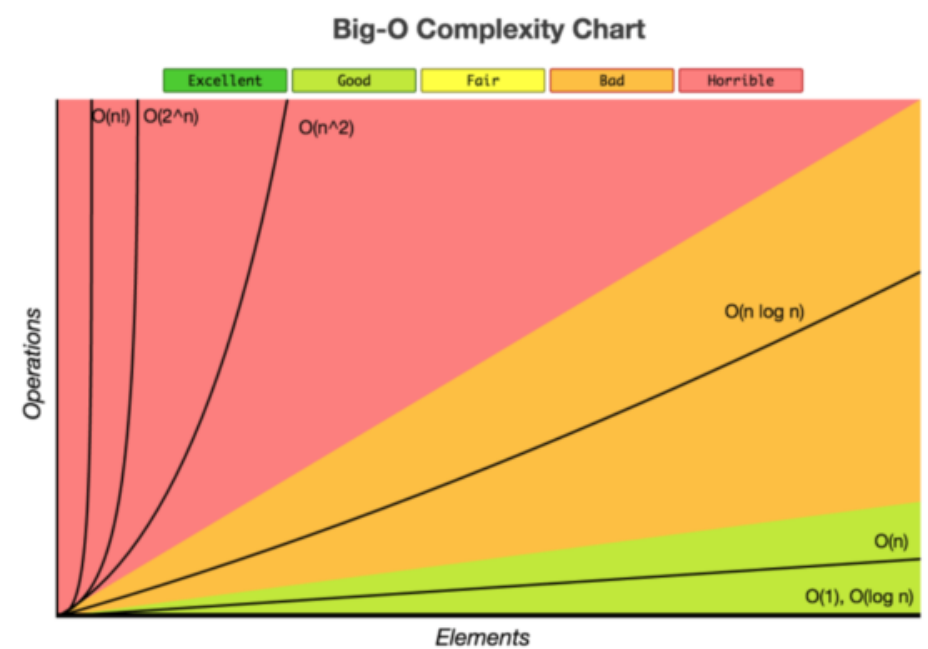


Figure 2: A visual representation of Big O Complexity [4].

The best case in Quicksort happens when the pivot selected consistently ends up in the middle of the list, splitting the list into two even sublists. This best case scenario for Quicksort runs in $\mathcal{O}(n \log n)$ time [1, 5].

The worst case in Quicksort happens when the pivot selection is consistently the smallest or greatest value in the list. This results in one sublist of size $n - 1$ and another of size 0. The algorithm then takes time proportional to n , then $n - 1$, then $n - 2$, which sums to $\Theta(n^2)$ time complexity [1, 5].

The average case in Quicksort is calculated by averaging the time complexity of all pivot positions. So, the average execution time across all possible random list arrangements strongly gravitates back to $\mathcal{O}(n \log n)$ [1, 5].

4 Implementation and Practical Considerations

For practicality, it is important to know the auxiliary space requirements of Quicksort. Let $S(n)$ be the auxiliary space required. The best-case recurrence is $S(n) = S(n/2) + \Theta(1)$, which yields $\Theta(\log n)$. The worst-case scenario is when recursion depth reaches n , the space complexity would be $\Theta(n)$ [1, 5].

Considering the time and space complexity, Quicksort would be best used on large data sets with limited amounts of RAM because Quicksort takes up very little auxiliary memory.

Quicksort is currently used wherever an unstable sort would be okay to use. Since it is very cache friendly, one can easily find examples. For example, in C++, `std::sort` uses Quicksort in its algorithm. Also, in commercial computing, Quicksort is especially important because it

is used in government systems for sorting large data sets. Now, below is the implementation in Python. Please note that this prints all of the steps taken by the algorithm [5].

```
1 import random
2 A = []
3 for i in range(0,15):
4     A.append(random.randint(0,100))
5 A = [5,8,3,4,10,7]
6 n = len(A)
7 print(A)
8
9 def quicksort(l,r,indent):
10     if l<r:
11         resultingpivotindex = partition(l,r,indent+2)
12         quicksort(l,resultingpivotindex-1,indent+2)
13         quicksort(resultingpivotindex+1,r,indent+2)
14     print(indent*'_ ' + 'Recombine: ' +str(A[l:r+1]))
15
16 def partition(l,r,indent):
17     print(indent*'_ ' + 'Subarray: ' + str(A[l:r+1]))
18     # To use a different pivotvalue
19     # swap it with A[r] here.
20     pivotvalue = A[r]
21     t = l
22     for i in range(l,r):
23         if A[i] <= pivotvalue:
24             temp = A[t]
25             A[t] = A[i]
26             A[i] = temp
27             t = t + 1
28     print(A)
29     temp = A[t]
30     A[t] = A[r]
31     A[r] = temp
32     print(indent*'_ ' + 'Pivot around final element.')
33     print(indent*'_ ' + 'Result: ' + str(A[l:r+1]))
34     return(t)
35
36 quicksort(0,n-1,0)
37 print(A)
```

References

- [1] David Gries. The basic quicksort algorithm. Technical report, Cornell University, 2018.
- [2] Medium User. Quicksort partitioning diagram. https://miro.medium.com/max/1400/1*aoXS7Bz8ZFEoQ-sq-UyysA.png, 2026. Accessed: September 19, 2026.
- [3] Stanford University. Tony Hoare: Quicksort, 2009. <https://cs.stanford.edu/people/eroberts/courses/soco/projects/2008-09/tony-hoare/quicksort.html>.

- [4] Tech with Davis. Big-O notation complexity chart. <https://techwithdavis.com/wp-content/uploads/2022/09/Big-O-notation-600x426.png>, 2022. Accessed: September 19, 2026.
- [5] Justin Wyss-Gallifent. CMSC 351: Quicksort. Technical report, University of Maryland, March 2025.